

Apache Kafka

0. **Установить Apache Kafka** (по аналогии с Hadoop: либо локально, либо Docker, либо на виртуальной машине / сервере) + не забыть установить необходимые модули для питончика (например `pip install kafka-python`). Задания делаем на пептоне

1. Producer / Consumer

- 1.1. Создать топик в кафке (например `orders`)
- 1.2. Сделать «производителя» (`producer`), который будет подключаться к кафке, отправлять сообщения в топик `orders`, отправлять 10 сообщений в формате Джейсон (JSON) `{ "order_id": 1, "price": 100 }`
- 1.3. Сделать «потребителя» (`consumer`), который будет читать сообщения из `order` и выводить их в терминал
- 1.4. Не забываем про команду `producer.flush()`

2. Много потребителей (consumer group)

- 2.1. Создать одного производителя, два потребителя, все должны быть в одной группе
- 2.2. Производитель должен отправлять 20 сообщений
- 2.3. Наблюдаем, как кафка распределит «partitions» (один потребитель прочитает часть, остальные прочитают остальное)
- 2.4. Повторить аналогично, но с 2 производителями и 3 потребителями

3. «Разделение» (Partitioning)

- 3.1. Производитель отправляет `{ "user_id": 15, "event": "click" }`
- 3.2. Добавляется ключ `key=str(user_id).encode()`
- 3.3. Пример:

```
producer.send(
    "events",
    key=str(user_id).encode(),
    value=data
)
```

Или одной строчкой, если лень внутри студента победила (9((
- 3.4. Проверить и убедиться, что сообщения с одинаковым `user_id` попадают в один раздел (`partition`)
- 3.5. Подсказка:

```
metadata = producer.send(...).get()
print(metadata.partition)
```

4. «Ordering»

- 4.1. Отправить с одинаковым ключом 1, 2, 3, 4, 5 — убедиться, что потребитель получил отправленное в том же порядке
- 4.2. Повторить действие, но ключи сделать разными и отправлять в разные разделы
- 4.3. Смотреть (внимательно) на порядок пришедших сообщений

5. Работа с «компенсациями» (offsets)

- 5.1. Мы должны понять разницу между автоматическими и ручными коммитами

- 5.2. Отключаем автокоммиты (`enable_auto_commit=False`)
- 5.3. После обработки сообщения делаем `consumer.commit()`
- 5.4. Пробуем два варианта:

<code>consumer.commit()</code>	<code>process(msg)</code>
<code>process(msg)</code>	<code>consumer.commit()</code>

- 5.5. Думаем

6. Работа с очередью недоставленных сообщений («Dead Letter Queue»)

- 6.1. Важно не терять проблемные события
- 6.2. Создаем два топика: `payments` и `payments-dlq`
- 6.3. Магия:

```
try:
    process(msg)

except Exception as e:
    producer.send(
        "payments-dlq",
        value=msg.value
    )
```

7. Великий замес между Exactly-once и At-least-once

- 7.1. Осознаем нормальность и причину возникновения дубликатов записей
- 7.2. Моделируем падение потребителя и повторное чтение сообщений по сценарию: Consumer читает сообщение. Обрабатывает. НЕ успевает закоммитить. Падает. После перезапуска сообщение читается снова.
- 7.3. В БД должны появиться дубликаты. Наблюдаем. Думаем.
- 7.4. Для хардкорщиков — страшная штука «`idempotency`», пробуем:


```
if event_id already processed:
    skip
```

8. Поточковая обработка

- 8.1. Мы должны освоить мастерство обработки потока событий в реальном времени и считывания «агрегатов»
- 8.2. Пусть producer генерирует вот это:


```
{
    "user_id": 15,
    "timestamp": "...",
    "event": "click"
}
```
- 8.3. Нужно считать количество кликов за последние 10 секунд
- 8.4. Используем следующие приблуды: `Kafka Streams` / `Faust` / `Spark Streaming`

9. Симбиоз Kafka и Spark

- 9.1. Нашаманить так, чтобы спарк читал топик кафки, агрегировал данные и выводил результаты
- 9.2. Подсказка:


```
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
```

```
.option("subscribe", "events") \  
.load()
```

- 9.3. Думаем, куда впихнуть `.option("checkpointLocation",
"/tmp/checkpoints")`

10.Проект «Real-time pipeline»

- 10.1. Берём архитектуру «логи > кафка > спарк > дэшборд» (для тех, кому очень скучно, можно elasticsearch, cliickhouse в последнем пункте)
- 10.2. С вас: генератор логов, приём событий кафкой, считывание спарком ошибок, RPS (запросы в секунду), формирование топа сервисов (больше всего событий, чаще падают, наибольшая нагрузка), вывод графаной, стримлит дэшборд и console sink